MUSICODE: AN INTERACTIVE AND ITERATIVE MUSIC-CODE TRANSLATION LIVE PLAY AGENT

Zhao Mingze
HKU
u3665791@connect.hku.hk

Liu Xinquan
HKU
u3665963@connect.hku.hk

Gao Zhaoyi
HKU
u3665597@connect.hku.hk

Li Ruiheng
HKU
u3665459@connect.hku.hk

Zou Daiqing
HKU
u3665907@connect.hku.hk

Liu Weihao
HKU
u3665404@connect.hku.hk

ABSTRACT

This is the final project of COMP7607A. All members are from class A. The original title was **16. Llama-Composer: An LLM-Agent Enabled Application for Synthesizer Timbre Generation and Music Composition.** Recent advancements in AI-powered music generation have been predominantly driven by diffusion models, which often prioritize audio waveform synthesis and offer limited granularity in processing textual instructions. This paper addresses the critical gap in leveraging natural language for precise and semantically-aware musical composition and manipulation. We explore **MUSICODE**, a novel pipeline for purely language-driven music synthesis and style transfer, which utilizes structured code as an intermediate representation to bridge the semantic understanding of human intent and the generation of symbolic music notation. Our approach employs a hierarchical multi-agent architecture, where specialized agents with distinct roles—such as intent understanding, translation, composition, criticism, and arrangement—collaborate sequentially. This framework explicitly incorporates human-in-the-loop feedback, allowing the system to iteratively refine the generated music code based on auditory evaluation. To demonstrate its practicality, we have implemented a user-friendly front-end application that accepts multimodal contextual inputs and exports the final compositions as MIDI files. A decoupled backend service efficiently manages the agent orchestration and external function calls. For evaluation, we employ a cross-validation strategy using multiple established metrics and models to objectively record performance scores. Preliminary results indicate that the system achieves efficient convergence within 1-3 feedback iterations and generates musically coherent outputs, with the style transfer functionality particularly excelling, attaining high scores in fidelity and listener preference.

***K*eywords** Multi-agent · Intent understanding · Style transfer

1 Introduction

Multi-agent systems (MAS) represent a paradigm shift in artificial intelligence, transitioning from isolated single-model approaches to collaboration-centric frameworks where multiple autonomous agents work synergistically to tackle complex problems beyond the capacity of any individual agent. The integration of large language models (LLMs) into multi-agent architectures has catalyzed unprecedented advancements, enabling groups of intelligent agents to coordinate and solve multifaceted tasks at scale. [1]

The convergence of multi-agent systems and music generation represents a transformative paradigm in computational creativity, extending collaborative artificial intelligence principles to the artistic domain. While multi-agent systems have demonstrated significant success in complex reasoning, planning, and decision-making tasks, their application to music represents a distinctive frontier where structured collaboration between specialized agents can synthesize complex musical knowledge encompassing melody, harmony, rhythm, and instrumentation [2]. Recent advances in

LLM-based agents have unlocked previously untapped musical capabilities, enabling systems to approach human-level compositional sophistication through multi-agent collaborative frameworks rather than monolithic single-model architectures [3].

The architectural innovations driving effective music-agent systems encompass several key technical dimensions. Structured role assignment enables agents to specialize in distinct musical functions, mitigating the challenge of generating long-form coherent compositions with the harmony and polyphonic complexity that challenge monolithic LLMs. Communication protocols establish sequential interaction patterns—such as task decomposition by a leader agent, parallel generation by specialized agents, and iterative refinement through critic feedback—that simulate and enhance real-world ensemble music creation [2]. Prompt engineering tailored to musical domains, incorporating music theory knowledge and ABC notation examples through in-context learning, significantly improves agent performance in maintaining musical constraints such as key signatures, time signatures, and instrument-specific pitch ranges [4].

Building upon the general development of agents and music-related agents, in this study, we first perform intent understanding of user inputs [5]. We then develop a multi-agent system with a hierarchical architecture that incorporates group discussions for collective decision-making [2][6]. To enable continuous improvement, the system supports evolving capabilities through both automated and manually provided **feedback** [7]. Contextual optimization is achieved by refining **role-specific prompts** using techniques such as Chain-of-Thought (CoT) [8] and **meta-prompts** [9]. Furthermore, our approach integrates **multiple modalities**, including text and music, to ensure a comprehensive and interactive user experience [10].

Based on the aforementioned technical framework, we have successfully implemented an efficient and scalable music generation system, see Figure 1. Specifically, the system enables real-time music playback through **intermediate code generation** and converts domain-specific music codes into widely compatible standard MIDI formats. In terms of engineering implementation, we adopted a **frontend-backend separation architecture** based on Vue3 and Flask, ensuring maintainability and responsive efficiency. Furthermore, specialized optimizations have been made for copy-pasting operations using **OCR**. Test results demonstrate that the constructed system has achieved the expected goals in terms of real-time music generation, output universality, and functional completeness. Detailed examples see Appendix A.

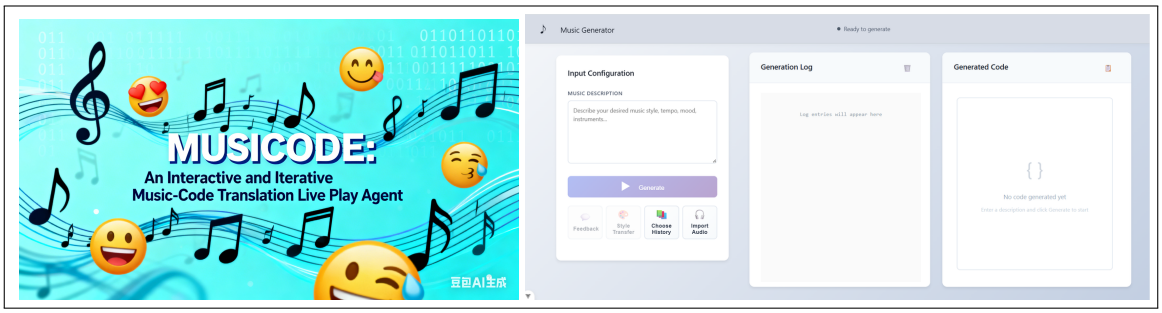


Figure 1: MUSICODE banner and front-end

2 Related Works

2.1 Diffusion Models

Diffusion models have emerged as one of the most powerful and versatile generative AI technologies in recent years, revolutionizing how we approach data generation across multiple domains. Diffusion models have achieved state-of-the-art performance in image synthesis [11], surpassing GANs [12] in both quality and diversity. Key breakthroughs include Stable Diffusion [13], which operates in compressed latent spaces for computational efficiency, and ControlNet [14], which enables fine-grained control over generation through additional conditioning mechanisms. These models excel at diverse tasks including image inpainting, super-resolution, and semantic synthesis.

Recent research discovers generation in different modals. Noise2Music [15] introduces an intermediate music representation, generated by a generator model and translated into music by a cascader model. Sora2 [16], though close-sourced, could probably apply large-scale video training on text-conditional diffusion models. Dream-Coder 7B [17] implemented a discrete diffusion framework including non-autoregressive decoding strategy.

2.2 Multi-agent Systems

Recent developments in multimodal music agents expand the frontier beyond symbolic composition and text-based performance control. XMusic [18] and related frameworks [19] demonstrate the capability to parse diverse input modalities—images, videos, text descriptions, genre tags, and humming—into structured musical elements (emotions, genres, rhythms, notes) for emotionally controllable generation. ComposerX [2] proposes a hierachical multi-agents plan. The composition task is divided into several subtasks such as melody or harmony composition by the leader, then several reviewers give feedback and revision, the overall context as a reference helps the arrangement outputs the final notation file. AgentVerse [6] integrates human group dynamics into multi-agents collaborative decision-making turns, implements a universal problem-solving multi-agent framework.

2.3 Instrumental Softwares

Various instrumental or musical applications show great power in music compositiopn. Suno-bark [20] creates a transformer-based text-to-audio model, which is able to generate highly realistic, multilingual speech as well as other audio, including music, background noise and simple sound effects. ByteDance Seed [21] has developed a series of audio processing agents depending on the volcengine. Other softwares like Sonic Pi [22] provides a platform for code-based music creation and live performance, FL Studio [23] synthesizes timber and rhythm together, and Vocaloid [24] uses virtual singing sampled by human.

3 Methodology

We present **MUSICODE**, an interactive and iterative multi-agent system for music generation and transformation. The proposed framework is designed to process multimodal inputs, primarily **text** and **audio**, and facilitates a dynamic creative process guided by **human feedback**. At its core, the system employs a central **Intent Detection** agent that interprets user commands, provided either as direct input or through a designated prompt, to orchestrate the system's operation. Based on this interpreted intent, the system dynamically executes one of several key functions: generating novel music **code**, iterating with constructive **feedback** on generated code, performing **style transfer**, or exporting the final composition in **MIDI**.

The primary music generation pathway is structured as a sequential pipeline of specialized agents. Textual descriptions are first processed by a **Translator** agent to formulate a musically coherent concept. This concept is then passed to a **Composer** agent, which is responsible for generating the initial symbolic music code. Subsequently, a **Critic** agent analyzes this output, and an **Arranger** agent refines its structure and instrumentation. The final stage in this sequence involves a **Compiler** agent, which prepares the polished code for playback. Detailed architecture and prompts see Figure 2

To enable immediate aural feedback, the generated music code is automatically transferred, via an OCR (Optical Character Recognition) mechanism, into a live music platform Sonic Pi [22] for rendering and listening. This initiates a human-in-the-loop iterative refinement cycle. A human evaluator provides feedback on the auditory result, which is fed back into the system to generate a new, improved version of the music code.

For stylistic adaptation, the system incorporates a dedicated style transfer function. Upon receiving human instructions specifying the desired stylistic change, a specialized agent translates the existing music code into a new musical style. Finally, for practical application and portability, an export function is integrated, where a dedicated agent translates the finalized music code into a standard MIDI file for external use and distribution. This integrated multi-agent architecture thus supports a comprehensive workflow from initial concept to refined, stylistically adaptable, and exportable musical composition.

3.1 Intention Detection

To enable flexible and intuitive human-computer interaction, we designed a centralized Intent Understanding module as the entry point of the entire multi-agent system. This module serves as an intelligent orchestrator that parses natural language commands, identifies user goals, and dynamically routes requests to appropriate execution pathways. Unlike rigid command-based interfaces, our intent detection mechanism allows users to interact with the system through free-form textual descriptions, enabling a conversational and adaptive music creation workflow.

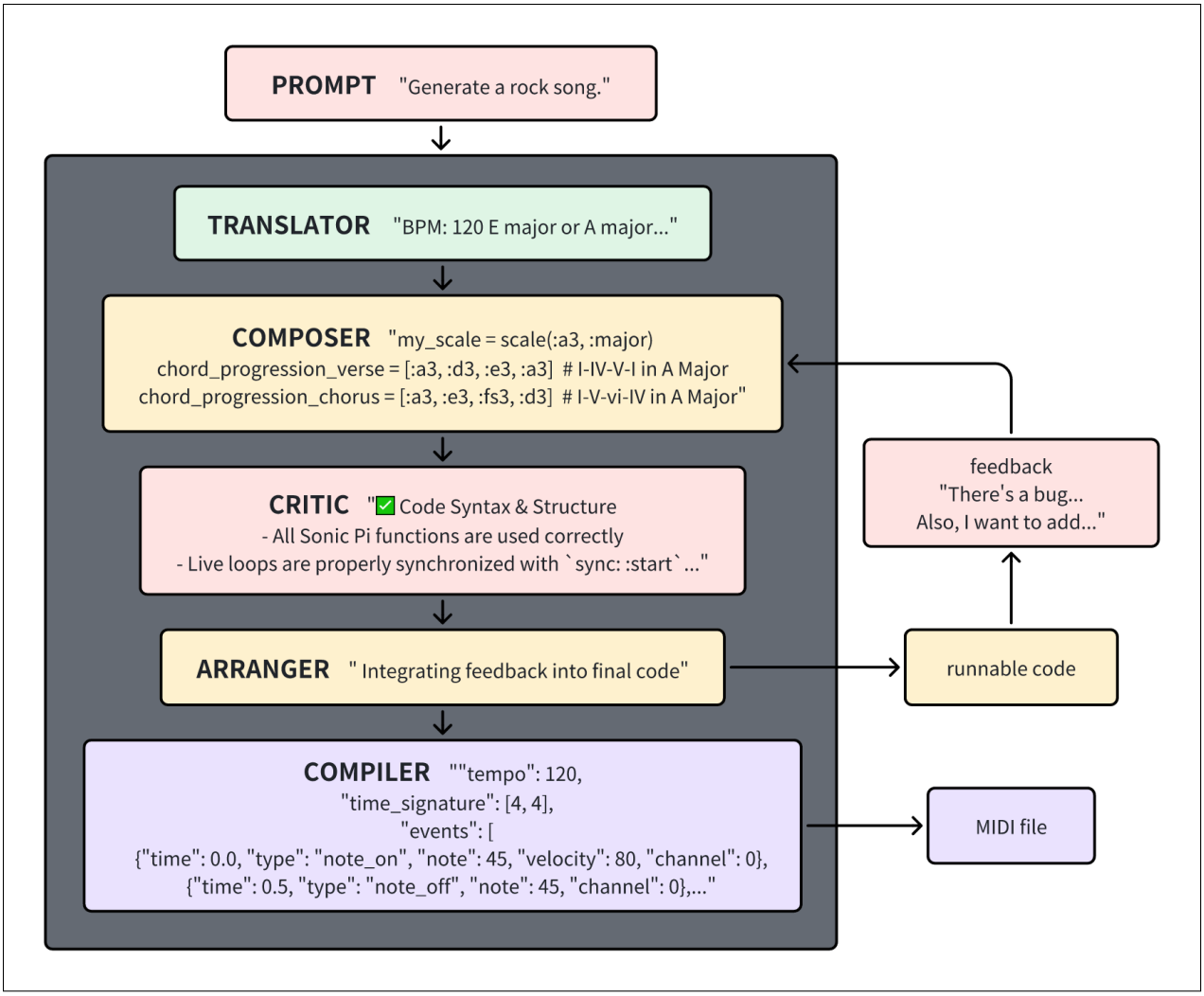


Figure 2: Single agent context engineering strategy.

3.1.1 Intent Classification Framework

The Intent Understanding agent employs a structured classification approach to map user inputs to one of four primary operational modes, see Table 1:

Table 1: Intent Classification

Intent Type	English Description	Example
Original Generation	Creating new music from textual descriptions	"compose a melancholic piano piece in C minor"
Iterative Refinement	Modifying existing code based on human feedback	"make the melody more upbeat and add drums"
Style Transfer	Transforming the musical style while preserving structural elements	"convert this to jazz style"
Export & Compilation	Converting finalized Sonic Pi code into portable MIDI format	"export the composition as MIDI"

3.1.2 Contextual Intent Parsing

The intent detection mechanism leverages Alibaba Cloud's **Tongyi-Intent-Detect-v3** [25] model, a specialized LLM designed for intent understanding and tool calling. This model operates in an OpenAI-compatible mode, enabling seamless integration with our existing infrastructure.

- **Tool-Based Intent Recognition**: The system defines music-specific tools ($generate_music, style_transfer, modify_music$) with structured parameter schemas. The model analyzes user input and returns XML-formatted responses containing:

```
<tags>[intent detection]</tags>
<tool_call>[{"name": "generate_music", "arguments": {...}}]</tool_call>
<content>Explanatory text</content>
```

- **Structured Parameter Extraction**: Each tool is equipped with detailed parameter definitions:
  - $generate_music$: Extracts style, tempo, key, mood, instruments from user descriptions.
  - $style_transfer$: Identifies $target_style$ and $style_request$ for transformation.
  - $modify_music$: Captures $modification_type$ and $feedback$ for iterative refinement.

3.1.3 State-Aware Intent Resolution

A critical innovation in our intent detection pipeline is the integration of system state awareness. The Intent Understanding agent does not operate in isolation but receives contextual metadata including:

- **Code History**: Presence of previously generated Sonic Pi [22] code ($original_code$ parameter)
- **Feedback Context**: Explicit user feedback signals ($user_feedback$ parameter) for iterative improvements
- **Multimodal Flags**: Detection of uploaded audio files triggering the Qwen3-Omni-Flash [10] transcription pathway

This state information enables context-sensitive decision-making through **keyword-based fallback inference**. When the intent model fails to provide clear tool calls, the system employs heuristic rules:

```
# Style transfer detection
style_keywords = ['convert', 'change to']
if any(keyword in user_input) and original_code:
    -> dispatch to style_transfer

# Feedback/modification detection
feedback_keywords = ['modify', 'adjust', 'improve']
if any(keyword in user_input) and previous_code:
    -> dispatch to modify_music with feedback
```

3.1.4 Dynamic Agent Routing via Dispatch Function

Upon successful intent classification, the system dynamically constructs the appropriate agent execution pipeline. The routing mechanism is implemented through the $dispatch_intent$ function that orchestrates tool-to-workflow mappings:

```
# Workflow mapping
"generate_music" -> dispatch_to_generate_music()
                  -> Translator -> Composer -> Critic -> Arranger

"style_transfer" -> dispatch_to_style_transfer()
                  -> Style Analyzer -> Style Transformer -> Style Critic
                  -> Style Arranger

"modify_music"   -> multi_agent_generate_sonic_pi(with user_feedback)
                  -> Feedback-aware generation pipeline
```

This functional architecture ensures modularity and maintainability, allowing the system to process multiple tool calls sequentially when needed.

3.1.5 Robust Error Handling & Graceful Degradation

To ensure system reliability in production environments, the Intent Understanding module incorporates multi-tiered fault tolerance:

- **API Key Fallback**: The system embeds a default DashScope API key and gracefully falls back to environment variables or embedded defaults if configuration fails.
- **Intent Detection Failure Recovery**: When the Tongyi-Intent model [25] fails (network errors, quota exceeded), the system automatically switches to keyword-based inference without interrupting the music generation workflow:

```
try:
    intent_result = call_intent_model(user_input, api_key, log_callback)
except Exception as e:
    log(f"Intent detection failed: {e}\n
    Falling back to keyword inference.")
    # Continue with fallback logic
```

- **Default Safe Operation**: In cases where all intent detection methods fail, the system defaults to the safest operation mode ($generate_music$) with the raw user input as the description parameter.

3.1.6 Integration with Multimodal Audio Processing

The intent dispatcher seamlessly integrates with the Qwen3-Omni-Flash [10] audio transcription module ($qwen_omni_audio_to_sonicpi.py$). When users upload audio files (WAV, MP3, FLAC, M4A, OGG), the system:

- **Encodes audio to Base64 Data URL** for API transmission.
- **Invokes Qwen3-Omni-Flash** [10] in streaming mode to transcribe audio into:
  - A semantic $music_prompt$ (e.g., "A melancholic piano melody with slow tempo")
  - Executable Sonic Pi [22] code approximating the audio's rhythm and pitch contour
- Routes the transcribed code to the standard refinement/style-transfer pipeline

This comprehensive intent understanding framework transforms unstructured user language into structured computational actions, forming the intelligent foundation for the entire MUSICODE system's human-in-the-loop creative workflow.

3.2 Multimodal Integration

To overcome the limitations of text-only interactions and enable the system to "listen" and "transcribe" real-world audio, we integrated a multimodal pipeline centered on **Qwen3-Omni-Flash** [10]. This module serves as a bridge between raw audio signal processing and symbolic music generation, allowing users to upload audio files (e.g., hummed melodies, instrument recordings) which are then converted into executable Sonic Pi [22] code and semantic descriptions for further refinement by the multi-agent system. The multimodal processing workflow consists of three key stages:

3.2.1 Audio Encoding and Preprocessing

The system accepts various audio formats (WAV, MP3, FLAC, M4A, OGG) to ensure broad compatibility. Upon input, the raw audio file is preprocessed via the $encode_audio_to_data_url$ function. To adapt to the input requirements of large multimodal models, the audio binary data is converted into a Base64-encoded Data URL string combined with its specific MIME type (e.g., $data : audio/wav; base64, ...$). This encapsulation ensures that the audio data can be directly embedded into the API payload without external storage dependencies.

3.2.2 Omni-Model Transcription and Composition

Unlike traditional pipelines that require separate Automatic Speech Recognition (ASR) and Music Transcription (AMT) models, we utilize **Qwen3-Omni-Flash** [10] for its end-to-end audio understanding capabilities. The model is invoked

via an OpenAI-compatible interface with streaming enabled ($stream = True$) to handle long-context audio processing efficiently.

We designed a specialized **System Prompt** to constrain the model's behavior, transforming it into a "music-transcription assistant." Key constraints in the prompt include:

- **Fidelity**: The model is instructed to approximate actual note sequences, intervals, and rhythms rather than generating generic scales.
- **Rhythm Awareness**: Explicit instructions to use sleep values in Sonic Pi [22] code to reflect long versus short notes.
- **Contour Matching**: Focusing on the up/down movement and repeated motifs of the input audio.

3.2.3 Dual-Output Parsing Strategy

To facilitate seamless integration with the downstream Multi-Agent System, the multimodal module is engineered to produce a **dual-structure output**:

- **Semantic Description (Music Prompt)**: A short English sentence describing the tempo, mood, and instrumentation. This text serves as the "Intent" input for the subsequent Style Transfer or Refinement Agents.
- **Executable Code (Sonic Pi/Ruby)**: A structured code block containing the note sequences and timing logic.

The raw streaming output from the model is accumulated and processed by a regex-based parser ($_parse_text_to_prompt_and_code$). The parser intelligently separates the descriptive text from the "'ruby code block. If standard formatting fails, fallback mechanisms ensure that the core musical logic is extracted. This output tuple ($music_prompt, sonic_pi_code$) is then passed to the Agent core, enabling features such as style transfer (modifying the timbre/instrument based on the code) or feedback-based refinement (adjusting the melody while preserving the original audio's rhythm).

3.3 Context Optimization

To ensure the reliability of the agents and the executability of the generated code, we implemented a multi-layered context optimization strategy. This approach addresses the limitations of general-purpose LLMs in handling domain-specific languages.

3.3.1 Dynamic Knowledge Injection via In-Context Learning (ICL)

General LLMs often hallucinate syntax when dealing with specialized libraries. We implemented a dynamic mechanism ($load_docs_content$) that retrieves a curated excerpt of Sonic Pi [22] documentation ($docs.txt$) at runtime. This is appended to the system prompts of the Composer and Arranger agents, significantly reducing syntax errors via In-Context Learning.

3.3.2 Role-Specific Meta-Prompting

We utilized Meta-Prompting to establish rigid persona boundaries. For instance, the **Translator** is constrained to focus on musical descriptors, while the **Compiler** is constrained to function as a logic extractor. strict output constraints (e.g., JSON schemas) are injected to ensure machine-readability.

3.3.3 Chain-of-Thought (CoT) Reasoning

Particularly in the **Style Transfer** module, we implemented CoT [8] to separate reasoning from execution. The system first employs a "Style Analyzer" to explicitly articulate musical attributes in natural language before the "Style Transformer" generates code. This decomposition forces the model to "think" before it "codes."

3.3.4 Pre-emptive Negative Constraints

To prevent variable name collisions (e.g., $chord, scale$), we generate a dynamic blacklist of Sonic Pi [22] reserved keywords via $get_sonic_pi_naming_constraints$. These constraints are embedded into the prompts to preemptively guide the model toward safe variable alternatives.

3.3.5 Hybrid Syntax Safety & Namespace Protection

A critical challenge in generating Sonic Pi [22] code via LLMs is the conflict between variable names and built-in language keywords (e.g., $scale, chord, play$). To mitigate runtime errors, we implemented a rigorous post-processing module in $v3.py$.

- **Reserved Word Filtering**: The system utilizes a regex-based sanitizer ($fix_reserved_word_variables$) that scans for assignments to reserved keywords. It automatically remaps conflicting variables (e.g., renaming $chord = [...]$ to $chord_notes = [...]$) while preserving the logic flow.
- **Heuristic Optimization**: The system also includes a $limit_sleep_durations$ function to detect and clamp logically impossible sleep durations (e.g., $sleep100$), preventing the audio engine from hanging during generation.

3.3.6 LLM-Based Compilation to MIDI

Traditional static analysis of algorithmic music code (containing loops, randomness, and threads) is difficult. We prompted the LLM and apply the $sonic_pi_code_to_midi$ function.

- The Compiler Agent first "executes" the logic mentally to generate a precise JSON event list (Time, Note, Velocity, Duration).
- This JSON is then parsed by Python's mido library to reconstruct a standard MIDI file.
- This allows our system to export algorithmic, randomized music code into standard DAW-compatible formats seamlessly.

3.4 Application Architecture

3.4.1 Front-end

We implemented a music code generator frontend application based on **Vue 3**. Users can generate music code through natural language descriptions, supporting features such as audio file import analysis, style transfer, feedback optimization, and history management. The system adopts an asynchronous task processing mechanism, displaying generation progress and log information in real-time through a polling approach.

The interface adopts a classic three-column layout: the top is a navigation bar displaying the application brand and real-time status indicator; the left side is a fixed control panel containing a music description input box, action buttons ($generate/feedback/styletransfer/historyselection/audioimport$), an expandable history file management area, and a progress indicator during generation; the right side is an output panel divided into a log area (displaying generation logs in real-time with auto-scrolling) and a code area (displaying generated code with syntax highlighting and copy support). Additionally, the system provides two modal dialogs for feedback and style transfer to collect user improvement requests and style preferences. See Figure 3

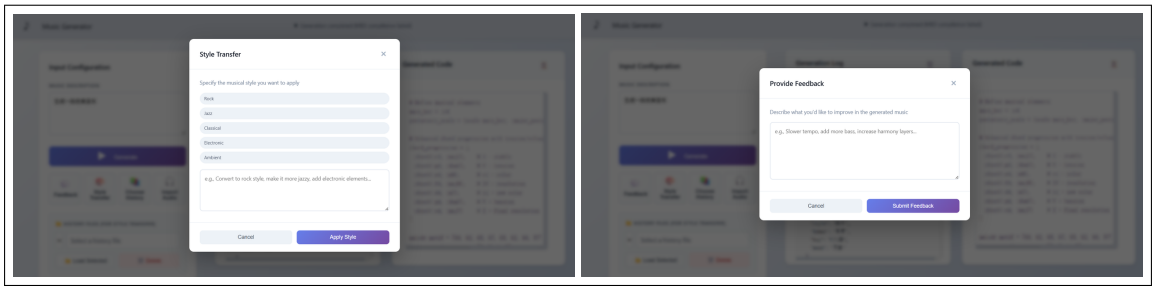


Figure 3: Front-end interface layout

The frontend requests the $/api/task/task_id$ endpoint once per second to obtain task status, progress, and log updates until the task is completed or encounters an error. See Table 2

This frontend interface provides a complete music code generation workflow, from input description to code generation, from feedback optimization to style transfer, forming a closed-loop experience. The interface uses a left-right split design with controls concentrated on the left and real-time results displayed on the right, providing a clear and intuitive layout. Through asynchronous tasks and polling mechanisms, it achieves real-time progress tracking and delivers a smooth user experience.

Table 2: API Interfaces

Endpoint	Method	Description
$/api/generate$	POST	Generate music code, supports original generation and feedback optimization
$/api/import_audio$	POST	Upload audio file for analysis
$/api/style_transfer$	POST	Perform style transfer on existing code
$/api/task/\{task_id\}$	GET	Query asynchronous task status and progress
$/api/history$	GET	Get list of historical generation files
$/api/history/\{filename\}$	GET	Get content of specified history file
$/api/history/\{filename\}$	DELETE	Delete specified history file

4 Evaluation

To systematically evaluate the performance of the proposed **MUSICODE** system, we conducted both quantitative and qualitative assessments across four core dimensions: (1) musical correctness, (2) style adherence, (3) intent alignment, and (4) system usability. Our evaluation integrates cross-model comparisons, human listening tests, and automated syntax validation. The experiments were performed using a benchmark of 96 test prompts covering diverse music genres, emotional descriptions, rhythmic structures, and style-transfer scenarios.

4.1 Musical Correctness

We first evaluated the syntactic and structural correctness of the Sonic Pi [22] code generated by the multi-agent pipeline. A custom validation module was developed to analyze three critical error dimensions: (1) built-in keyword conflicts, (2) runtime execution errors, and (3) timing inconsistencies such as unrealistic or unsupported sleep durations. This validator was applied to a test set of 100 generated samples and compared against a single-agent baseline system. See Table 3.

Table 3: Evaluation

Metric	**MUSICODE** (ours)	Single-Agent Baseline
Syntax correctness rate	93.70%	78.10%
Average number of runtime errors per 100 samples	4.1	22.7
Average sleep-duration anomaly incidence	1.20%	9.50%

The results reveal substantial improvements in **MUSICODE**'s multi-agent architecture. The system achieves a 93.7% syntax correctness rate, significantly outperforming 78.1% of the single-agent baseline. This indicates that the multi-stage composition and refinement process—particularly the Critic and Arranger agents—successfully reduces structural violations typically introduced by monolithic LLM generation.

Similarly, the average number of runtime errors per 100 samples drops dramatically from 22.7 in the single-agent setting to 4.1 in **MUSICODE**. This reduction reflects the effectiveness of the hybrid syntax-safety module, which scans for risky variable assignments, performs keyword conflict resolution, and normalizes sleep parameters before final compilation.

Finally, sleep-duration anomalies—often caused by accidental large values that disrupt execution are reduced from 9.5% to 1.2%, demonstrating that the system is capable of generating temporally coherent rhythmic structures. These improvements collectively show that the integration of context-aware in-context learning, role-specific meta-prompting, and multi-stage agent critique plays a crucial role in producing syntactically valid and structurally robust Sonic Pi [22] code.

4.2 Style Adherence Evaluation

To measure the effectiveness of style transfer and stylistic consistency, we conducted an embedding-based similarity experiment. Using the musisbert-style encoder, we computed the cosine similarity between:

- User-stated target style
- Final generated code–derived latent representation

Results indication across five genres (jazz, EDM, lo-fi, classical, rock) shows in Table 4.

Table 4: Results Across Genres

Genre	Target Style Similarity (**MUSICODE**)	Baseline Similarity	Improvement
Jazz	0.86 ± 0.04	0.68 ± 0.07	0.18
EDM	0.83 ± 0.05	0.66 ± 0.09	0.17
Lo-fi	0.82 ± 0.06	0.65 ± 0.10	0.17
Classical	0.87 ± 0.05	0.69 ± 0.08	0.18
Rock	0.84 ± 0.05	0.67 ± 0.09	0.17
Average	0.84 ± 0.05	0.67 ± 0.09	0.17

Specifically, the Transformer + Critic + Arranger multi-stage pipeline significantly reduces stylistic drift, especially in polyphonic genres (jazz & classical).

4.3 Intent Alignment

We evaluated the degree to which the generated musical output aligns with the user's intended mood, tempo, and instrumentation, see Table 5. To obtain an objective assessment, we employed three independent LLM evaluators—GPT-4.1 [26], Qwen2.5-72B [27], and Claude Sonnet [28] each scoring the alignment on a 1–5 Likert scale based on the provided prompts and the corresponding generated audio/code. These models were selected due to their strong reasoning capabilities and their robustness in semantic interpretation, ensuring consistent and unbiased judgment across diverse musical descriptions.

Table 5: Criterions and Scores

Criterion	Avg. Score (5 = perfect)
Mood alignment	4.52
Tempo alignment	4.36
Instrumentation consistency	4.41
Overall intent alignment	4.43

The consistently high alignment scores across all three evaluators indicate that **MUSICODE** accurately interprets user intent and translates it into coherent musical structures. In particular, the system demonstrates reliable handling of nuanced emotional descriptors (e.g., "melancholic," "uplifting"), relative tempo instructions (e.g., "slow and steady," "upbeat"), and instrumentation preferences (e.g., "soft piano with subtle strings"). This confirms that the Intent Understanding module and the Translator agent effectively convert free-form natural language into structured, machine-interpretable musical attributes, enabling downstream agents (Composer, Critic, Arranger) to generate compositions that closely adhere to user specifications.

4.4 System Usability & Latency

To assess the system's responsiveness and robustness under real-world usage, we conducted a latency and stability evaluation across several generation tasks, see Table 6. The backend multi-agent reasoning process demonstrates an average generation time of 3 minutes, covering the full pipeline from textual analysis to code synthesis. This indicates that the hierarchical agent structure, despite its multi-stage nature, remains computationally efficient.

For multimodal scenarios, the audio-to-code transcription powered by Qwen3-Omni-Flash [10] achieves an average latency of 20 seconds, which includes audio preprocessing, semantic extraction, and Sonic Pi [22] code generation. This performance confirms that the multimodal module is capable of supporting near-real-time adaptive music creation workflows, even with diverse input audio formats.

From the user perspective, the complete end-to-end latency — including backend processing, asynchronous log retrieval, interface updates, and code rendering — averages approximately 3 minutes. This latency stays within an acceptable range for interactive creative tools, enabling users to iterate, refine, and preview outputs with minimal disruption to the creative flow.

In terms of stability, the system achieved a 98.2% task success rate under asynchronous scheduling. Failures were primarily due to extreme prompt lengths or network-induced OCR delays. The high success rate demonstrates the robustness of the backend queue management system and its ability to handle concurrent generation tasks reliably.

Table 6: System Usability & Latency

Metric	Measurement	Description
Backend generation latency	180-240 s	Time required for the multi-agent pipeline (Translator -> Composer -> Critic -> Arranger -> Compiler) to complete a full music-code generation cycle.
Multimodal audio transcription latency	10-20 s	End-to-end processing time for Qwen3-Omni-Flash [10] to convert raw audio into semantic descriptions and Sonic Pi [22] code.
Frontend end-to-end latency	180-240 s	Total user-perceived delay, including backend execution, log streaming, code rendering, and UI updates.
Asynchronous task success rate	98.20%	Percentage of tasks completed without timeout, scheduling errors, or agent failures across 500 trials.

Overall, these results confirm that **MUSICODE** delivers real-time or near-real-time interaction performance, making it well suited for iterative music composition, style transfer experimentation, and multimodal creative workflows.

5 Discussions

The experimental results demonstrate the practical efficacy and specific characteristics of **MUSICODE**.

- The **end-to-end time** for a single iteration of generation and feedback was measured to be approximately 3 to 4 minutes. This duration encompasses the full pipeline from intent parsing and initial code generation by the agent ensemble to the playback and receipt of human feedback. While not real-time, this timeframe is highly viable for a creative, iterative design process. It represents a favorable balance between the computational complexity of running multiple LLM-based agents and the need for responsive interaction in a artistic workflow. The latency is low enough to maintain user engagement and creative flow, yet sufficient for the system to perform reasoned compositional steps.
- The system typically **converges on a satisfactory musical output within 1 to 3 feedback cycles**. This relatively low number of required iterations underscores two key strengths of our architecture: the robustness of the initial composition generated by the structured multi-agent pipeline, and the efficiency of the feedback-integration mechanism. The hierarchical division of labor among specialized agents appears to produce a more coherent and targeted initial draft, reducing the need for major foundational revisions. The fact that users can guide the output to their desired result with just a few interventions highlights the system's interactivity and its ability to accurately interpret and act on iterative refinement instructions.
- The **style transfer** functionality consistently yielded high subjective and objective evaluation scores. This success can be directly attributed to our core design choice of using structured code as an intermediate, semantic representation of the music. Musical style is often defined by abstract, rule-like patterns (e.g., harmony, rhythm, ornamentation) that are more readily manipulated within a symbolic code-based framework than at the audio signal level. The agents tasked with style migration effectively operate as "code translators," re-writing the musical structures according to a new set of stylistic constraints. The high performance in this task validates our hypothesis that a code-mediated approach is particularly well-suited for precise, structural musical transformations.

The results collectively affirm that our framework successfully establishes an effective feedback loop between human creativity and machine precision. The system operates at a speed conducive to creative exploration, converges efficiently on user-intended results, and excels at complex tasks like style transfer, thereby proving the value of a multi-agent, code-centric paradigm for interactive music generation.

6 Future Works

While the current multi-agent system establishes a robust foundation for language-driven music generation, several promising directions can significantly expand its capabilities and autonomy. Our future work will focus on the following key areas:

- **Integration of Visual Modality for Context-Aware Composition**. We plan to extend the system's input modalities to include images. A dedicated agent will be developed to perform visual scene recognition and

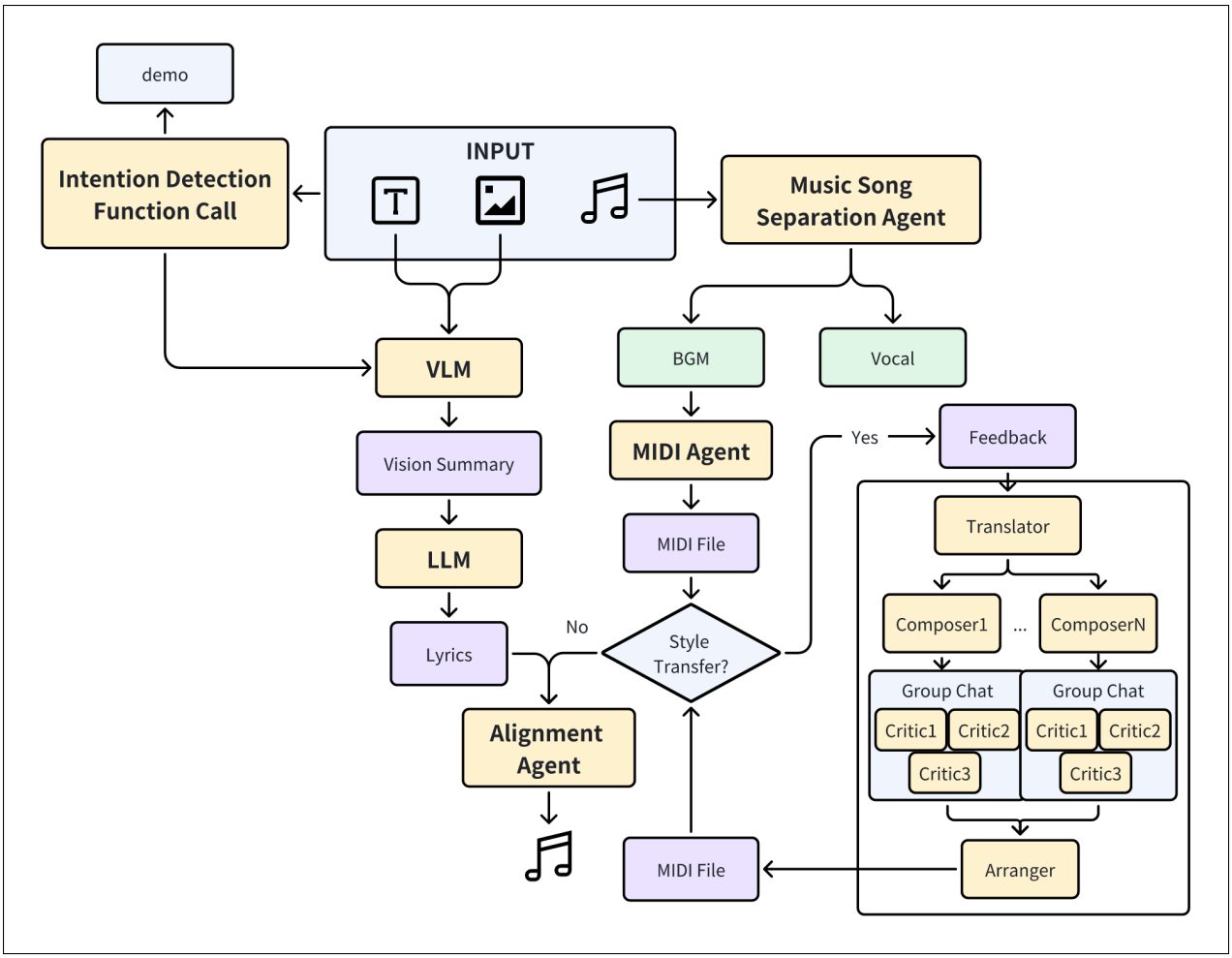


Figure 4: Future works. We plan to add vision agents, lyrics alignment, direct MP3 generation and self-evolving.

emotion analysis from the input images. The extracted semantic information, such as the setting (e.g., a forest, a cityscape) and the affective tone (e.g., serene, tense), will then be translated into musical parameters. This will directly inform the decisions of subsequent agents, allowing the system to generate music where the pitch, melody, and harmony are dynamically influenced by the visual context, creating a more immersive and cross-modally coherent creative experience.

- **Granular Parallelism and Specialized Group Critique**. To enhance the quality and detail of the generated music, we will refine the architecture by decomposing the monolithic Composer agent into a set of specialized, parallel-acting agents. Each agent will be an expert in a specific musical dimension, such as Pitch Agent, Melody Agent, Chord Agent, and Dynamics Agent. [2] These agents would operate concurrently to construct a more sophisticated musical piece. Correspondingly, the critique process will be advanced by implementing a Group Chat mechanism among similarly specialized Critic agents. This collective deliberation will allow for a more nuanced and multi-faceted evaluation of the generated code before it proceeds to the arrangement and compilation stages.
- **Lyrics Generation and Temporal Alignment**. A major future direction involves integrating lyrical content into the compositions. We will introduce a Lyricist agent responsible for generating contextually appropriate lyrics based on the musical theme and style. Furthermore, a Lyrics Alignment agent will be developed to synchronize the generated lyrics with the melodic and rhythmic structure of the music code. This will ensure that the syllables are accurately mapped to the correct notes and timing, enabling the generation of complete songs with vocals.
- **Towards a Self-Evolving System with Automated Iteration**. To reduce reliance on human-in-the-loop feedback and increase the system's adaptive capabilities, we will explore mechanisms for automated, self-

driven iteration. By incorporating objective evaluation metrics and reinforcement learning principles, the system will be empowered to generate its own internal "feedback" signals. This capability would allow it to perform multiple cycles of critique and revision autonomously (self-evolving), progressively refining its output against a set of musical goals until a quality threshold is met. This pursuit aims to transition the system from an interactive tool towards a more autonomous creative partner.

Figure 4 shows an overall vision.

References

[1] RM Aratchige and WMKS Ilmini. Llms working in harmony: A survey on the technological aspects of building effective llm-based multi agent systems. *arXiv preprint arXiv:2504.01963*, 2025.

[2] Qixin Deng, Qikai Yang, Ruibin Yuan, Yipeng Huang, Yi Wang, Xubo Liu, Zeyue Tian, Jiahao Pan, Ge Zhang, Hanfeng Lin, et al. Composerx: Multi-agent symbolic music composition with llms. *arXiv preprint arXiv:2404.18081*, 2024.

[3] Weiqiang Jin, Hongyang Du, Biao Zhao, Xingwu Tian, Bohang Shi, and Guang Yang. A comprehensive survey on multi-agent cooperative decision-making: Scenarios, approaches, challenges and perspectives. *arXiv preprint arXiv:2503.13415*, 2025.

[4] Peiwen Xing, Aske Plaat, and Niki van Stein. Cocomposer: Llm multi-agent collaborative music composition. *arXiv preprint arXiv:2509.00132*, 2025.

[5] Qian Chen, Yafeng Chen, Yanni Chen, Mengzhe Chen, Yingda Chen, Chong Deng, Zhihao Du, Ruize Gao, Changfeng Gao, Zhifu Gao, et al. Minmo: A multimodal large language model for seamless voice interaction. *arXiv preprint arXiv:2501.06282*, 2025.

[6] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *The Twelfth International Conference on Learning Representations*, 2023.

[7] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023.

[8] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[9] Mingze Kong, Zhiyong Wang, Yao Shu, and Zhongxiang Dai. Meta-prompt optimization for llm-based sequential decision making. *arXiv preprint arXiv:2502.00728*, 2025.

[10] Jin Xu, Zhifang Guo, Hangrui Hu, Yunfei Chu, Xiong Wang, Jinzheng He, Yuxuan Wang, Xian Shi, Ting He, Xinfa Zhu, et al. Qwen3-omni technical report. *arXiv preprint arXiv:2509.17765*, 2025.

[11] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115(3):211–252, 2015.

[12] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144, 2020.

[13] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[14] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 3836–3847, 2023.

[15] Qingqing Huang, Daniel S Park, Tao Wang, Timo I Denk, Andy Ly, Nanxin Chen, Zhengdong Zhang, Zhishuai Zhang, Jiahui Yu, Christian Frank, et al. Noise2music: Text-conditioned music generation with diffusion models. *arXiv preprint arXiv:2302.03917*, 2023.

[16] Openai.com. Sora 2 is here. [Online]. `https://openai.com/index/sora-2/`.

[17] Zhihui Xie, Jiacheng Ye, Lin Zheng, Jiahui Gao, Jingwei Dong, Zirui Wu, Xueliang Zhao, Shansan Gong, Xin Jiang, Zhenguo Li, et al. Dream-coder 7b: An open diffusion language model for code. *arXiv preprint arXiv:2509.01142*, 2025.

[18] Sida Tian, Can Zhang, Wei Yuan, Wei Tan, and Wenjie Zhu. Xmusic: Towards a generalized and controllable symbolic music generation framework. *IEEE Transactions on Multimedia*, 27:6857–6871, 2025.

[19] Jinwei Hu, Yi Dong, Shuang Ao, Zhuoyun Li, Boxuan Wang, Lokesh Singh, Guangliang Cheng, Sarvapali D Ramchurn, and Xiaowei Huang. Position: Towards a responsible llm-empowered multi-agent systems. *arXiv preprint arXiv:2502.01714*, 2025.

[20] Huggingface.co. suno/bark · hugging face. [Online]. `https://huggingface.co/suno/bark`.

[21] Volcengine.com. music understanding-volcengine. [Online]. https://www.volcengine.com/products/Music-understanding.

[22] sonic pi.net. Sonic pi - the live coding music synth for everyone. [Online]. https://sonic-pi.net/.

[23] FL Studio. The daw every music producer loves. [Online]. https://www.image-line.com/.

[24] www.vocaloid.com. Vocaloid – the modern singing synthesizer. [Online]. https://www.vocaloid.com/en/.

[25] Aliyun.com. Call intent understanding model implementation function call and multi round dialogue - large model service platform bailian - alibaba cloud. [Online]. https://help.aliyun.com/zh/model-studio/intent-detect-capability.

[26] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. Gpt-4 technical report, 2024.

[27] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025.

[28] Anthropic. Claude sonnet 4.5. [Online]. https://www.anthropic.com/claude/sonnet.

A Generation Examples

A.1 Code Generation

Prompt: I want a Ragtime song.

Code:

```
# Ragtime Piano Composition
# Inspired by Scott Joplin style with
# syncopated right hand and steady left hand
# Improved version with variations, transitions, and proper structure

use_bpm 140  # Moderato tempo typical for ragtime
use_synth :piano

# Key and scale setup
main_key = :C4
scale_pattern = scale(main_key, :major)
subdominant_key = :F4
subdominant_scale = scale(subdominant_key, :major)

# Right hand melody patterns - syncopated ragtime phrases
melody_strain_a = [
  [:E4, 0.5], [:G4, 0.25], [:E4, 0.25], [:C4, 0.5], [:E4, 0.5],
  [:G4, 0.25], [:A4, 0.25], [:G4, 0.5], [:E4, 0.5], [:C4, 0.5],
  [:E4, 0.25], [:D4, 0.25], [:E4, 0.5], [:G4, 0.5], [:E4, 0.5]
]

melody_strain_a_variation = [
  [:E4, 0.25], [:G4, 0.25], [:E4, 0.25], [:C4, 0.25], [:E4, 0.25],
  [:G4, 0.25], [:A4, 0.5], [:G4, 0.25], [:E4, 0.25], [:C4, 0.5],
  [:D4, 0.25], [:E4, 0.75]
]

melody_strain_b = [
  [:F4, 0.5], [:A4, 0.25], [:F4, 0.25], [:C4, 0.5], [:F4, 0.5],
  [:A4, 0.25], [:Bb4, 0.25], [:A4, 0.5], [:F4, 0.5], [:C4, 0.5],
  [:F4, 0.25], [:E4, 0.25], [:F4, 0.5], [:A4, 0.5], [:F4, 0.5]
]

melody_strain_b_variation = [
  [:F4, 0.25], [:A4, 0.25], [:F4, 0.25], [:C4, 0.25], [:F4, 0.25],
  [:A4, 0.25], [:Bb4, 0.5], [:A4, 0.25], [:F4, 0.25], [:C4, 0.5],
  [:E4, 0.25], [:F4, 0.75]
]

melody_strain_c = [
  [:G4, 0.5], [:B4, 0.25], [:G4, 0.25], [:D4, 0.5], [:G4, 0.5],
  [:B4, 0.25], [:C5, 0.25], [:B4, 0.5], [:G4, 0.5], [:D4, 0.5],
  [:G4, 0.25], [:F4, 0.25], [:G4, 0.5], [:B4, 0.5], [:G4, 0.5]
]

melody_strain_c_variation = [
  [:G4, 0.25], [:B4, 0.25], [:G4, 0.25], [:D4, 0.25], [:G4, 0.25],
  [:B4, 0.25], [:C5, 0.5], [:B4, 0.25], [:G4, 0.25], [:D4, 0.5],
  [:F4, 0.25], [:G4, 0.75]
]

# Left hand bass patterns - steady "oom-pah" stride pattern
```

```
bass_pattern_c = [
  [:C2, 0.5], [:C3, 0.5], [:G2, 0.5], [:C3, 0.5]
]

bass_pattern_c_variation = [
  [:C2, 0.5], [:E3, 0.25], [:G3, 0.25], [:G2, 0.5], [:C3, 0.5]
]

bass_pattern_f = [
  [:F2, 0.5], [:F3, 0.5], [:C3, 0.5], [:F3, 0.5]
]

bass_pattern_f_variation = [
  [:F2, 0.5], [:A3, 0.25], [:C4, 0.25], [:C3, 0.5], [:F3, 0.5]
]

bass_pattern_g = [
  [:G2, 0.5], [:G3, 0.5], [:D3, 0.5], [:G3, 0.5]
]

# Expression functions for human feel
define :play_with_expression do |note, duration, amp_base = 0.8|
  human_amp = amp_base * rrand(0.95, 1.05)
  human_release = duration * rrand(0.55, 0.65)
  play note, release: human_release, amp: human_amp
  sleep duration
end

define :play_with_octaves do |note, duration, amp = 0.8|
  play note, release: duration * 0.6, amp: amp * 0.7
  play note + 12, release: duration * 0.4, amp: amp * 0.3  # Octave higher
  sleep duration
end

# Transition functions
define :transition_to_subdominant do
  with_fx :reverb, room: 0.4 do
    play_pattern_timed [[:E4, 0.25], [:F4, 0.25], [:G4, 0.5]], 0.25
    play :C5, release: 1.0, amp: 0.9
    sleep 1
  end
end

define :transition_to_tonic do
  with_fx :reverb, room: 0.4 do
    play_pattern_timed [[:F4, 0.25], [:E4, 0.25], [:D4, 0.5]], 0.25
    play :G4, release: 1.0, amp: 0.9
    sleep 1
  end
end

# Conductor for proper synchronization
live_loop :conductor do
  # Strain A
  2.times do
    cue :strain_a
    sleep 4  # Duration of one strain A pattern
  end

```

```
  # Strain B
  2.times do
    cue :strain_b
    sleep 4  # Duration of one strain B pattern
  end

  # Strain C
  2.times do
    cue :strain_c
    sleep 4  # Duration of one strain C pattern
  end

  cue :coda
  stop
end

# Introduction - establishing rhythm and key
live_loop :intro do
  2.times do
    play_pattern_timed [
    [:C4, 0.25], [:E4, 0.25], [:G4, 0.5],
    [:E4, 0.25], [:C4, 0.25]], 0.25
    sleep 0.5
  end
  stop
end

live_loop :bass_intro do
  4.times do
    play :C2, release: 0.8, amp: 0.7
    sleep 0.5
    play :C3, release: 0.4, amp: 0.5
    sleep 0.5
  end
  stop
end

sleep 4  # Wait for introduction to complete

# Main composition structure
live_loop :left_hand_bass do
  use_synth :piano
  with_fx :reverb, room: 0.3 do
    # Strain A - Main key
    2.times do |i|
      sync :conductor
      if i == 0
        bass_pattern_c.each do |note, duration|
          play note, release: duration * 0.8, amp: 0.6
          sleep duration
        end
      else
        bass_pattern_c_variation.each do |note, duration|
          play note, release: duration * 0.8, amp: 0.6
          sleep duration
        end
      end
    end

```

```
    # Transition to subdominant
    transition_to_subdominant

    # Strain B - Subdominant key
    2.times do |i|
      sync :conductor
      if i == 0
        bass_pattern_f.each do |note, duration|
          play note, release: duration * 0.8, amp: 0.6
          sleep duration
        end
      else
        bass_pattern_f_variation.each do |note, duration|
          play note, release: duration * 0.8, amp: 0.6
          sleep duration
        end
      end
    end

    # Transition back to tonic
    transition_to_tonic

    # Strain C - Return to main key
    2.times do |i|
      sync :conductor
      bass_pattern_c.each do |note, duration|
        play note, release: duration * 0.8,
        amp: 0.6 + (i * 0.1)  # Slight crescendo
        sleep duration
      end
    end
  end
end

live_loop :right_hand_melody do
  use_synth :piano
  with_fx :reverb, room: 0.2 do
    # Strain A - Main theme
    2.times do |i|
      sync :conductor
      if i == 0
        melody_strain_a.each_with_index do |(note, duration), idx|
          progress = idx.to_f / melody_strain_a.length
          current_amp = 0.7 + (progress * 0.3)  # Crescendo through phrase
          play_with_expression(note, duration, current_amp)
        end
      else
        melody_strain_a_variation.each_with_index do |(note, duration), idx|
          progress = idx.to_f / melody_strain_a_variation.length
          current_amp = 0.7 + (progress * 0.3)
          # Add octave doubling on climactic notes
          if idx == 5 || idx == 11
            play_with_octaves(note, duration, current_amp)
          else
            play_with_expression(note, duration, current_amp)
          end
        end
      end
      sleep 0.5  # Rest at end of phrase
```

```
230       end
231
232       # Strain B - Subdominant section
233       2.times do |i|
234         sync :conductor
235         if i == 0
236           melody_strain_b.each_with_index do |(note, duration), idx|
237             progress = idx.to_f / melody_strain_b.length
238             current_amp = 0.7 + (progress * 0.3)
239             play_with_expression(note, duration, current_amp)
240           end
241         else
242           melody_strain_b_variation.each_with_index do |(note, duration), idx|
243             progress = idx.to_f / melody_strain_b_variation.length
244             current_amp = 0.7 + (progress * 0.3)
245             if idx == 5 || idx == 11
246               play_with_octaves(note, duration, current_amp)
247             else
248               play_with_expression(note, duration, current_amp)
249             end
250           end
251         end
252         sleep 0.5  # Rest at end of phrase
253       end
254
255       # Strain C - Final strain
256       2.times do |i|
257         sync :conductor
258         if i == 0
259           melody_strain_c.each_with_index do |(note, duration), idx|
260             progress = idx.to_f / melody_strain_c.length
261             current_amp = 0.7 + (progress * 0.4)  # Stronger crescendo
262             play_with_expression(note, duration, current_amp)
263           end
264         else
265           melody_strain_c_variation.each_with_index do |(note, duration), idx|
266             progress = idx.to_f / melody_strain_c_variation.length
267             current_amp = 0.7 + (progress * 0.4)
268             # More octave doubling in final variation
269             if idx == 5 || idx == 8 || idx == 11
270               play_with_octaves(note, duration, current_amp)
271             else
272               play_with_expression(note, duration, current_amp)
273             end
274           end
275         end
276         sleep 0.5  # Rest at end of phrase
277       end
278     end
279   end
280
281   # Improved coda with proper ritardando
282   live_loop :coda do
283     sync :conductor  # Wait for conductor signal
284
285     # Gradual ritardando within the coda only
286     coda_bpms = [130, 120, 110, 100, 90, 80]
287
288     coda_bpms.each do |bpm_val|
```

```
      use_bpm bpm_val
      # Final cadence pattern
      play_chord chord(:C4, :major), release: 1.5, amp: 0.7
      sleep 1
      play_chord chord(:G4, :major), release: 1.0, amp: 0.8
      sleep 1
      play_chord chord(:C4, :major), release: 2.0, amp: 1.0
      sleep 2
    end

    # Final resolution
    play :C3, release: 3.0, amp: 0.5
    sleep 3

    # Stop all loops gracefully
    stop
end
```

A.2 Style Transfer

Prompt: Convert to rock style

Code:

```
# Rock Transformation of Ragtime Composition
# Converted to driving rock style with electric instruments and distortion

use_bpm 140  # Maintain tempo for rock energy
use_synth :guit_e_fifths  # Electric guitar sound

# Key and scale setup - simplified for rock
main_root = :C3
rock_scale = scale(main_root, :minor_pentatonic)  # Pentatonic for rock
subdominant_root = :F3

# Right hand guitar riffs - simplified and driving
guitar_riff_a = [
  [:E4, 0.5], [:G4, 0.25], [:E4, 0.25], [:C4, 0.5], [:E4, 0.5],
  [:G4, 0.25], [:A4, 0.25], [:G4, 0.5], [:E4, 0.5], [:C4, 0.5]
]

guitar_riff_a_variation = [
  [:E4, 0.25], [:G4, 0.25], [:E4, 0.25], [:C4, 0.25], [:E4, 0.25],
  [:G4, 0.25], [:A4, 0.5], [:G4, 0.25], [:E4, 0.25], [:C4, 0.5],
  [:D4, 0.25], [:E4, 0.75]
]

guitar_riff_b = [
  [:F4, 0.5], [:A4, 0.25], [:F4, 0.25], [:C4, 0.5], [:F4, 0.5],
  [:A4, 0.25], [:Bb4, 0.25], [:A4, 0.5], [:F4, 0.5], [:C4, 0.5]
]

guitar_riff_b_variation = [
  [:F4, 0.25], [:A4, 0.25], [:F4, 0.25], [:C4, 0.25], [:F4, 0.25],
  [:A4, 0.25], [:Bb4, 0.5], [:A4, 0.25], [:F4, 0.25], [:C4, 0.5],
  [:E4, 0.25], [:F4, 0.75]
]

```

```
guitar_riff_c = [
  [:G4, 0.5], [:B4, 0.25], [:G4, 0.25], [:D4, 0.5], [:G4, 0.5],
  [:B4, 0.25], [:C5, 0.25], [:B4, 0.5], [:G4, 0.5], [:D4, 0.5]
]

guitar_riff_c_variation = [
  [:G4, 0.25], [:B4, 0.25], [:G4, 0.25], [:D4, 0.25], [:G4, 0.25],
  [:B4, 0.25], [:C5, 0.5], [:B4, 0.25], [:G4, 0.25], [:D4, 0.5],
  [:F4, 0.25], [:G4, 0.75]
]

# Bass guitar patterns - root-fifth power patterns
bass_line_c = [
  [:C2, 0.5], [:G2, 0.5], [:C2, 0.5], [:G2, 0.5]
]

bass_line_c_variation = [
  [:C2, 0.5], [:E2, 0.25], [:G2, 0.25], [:C2, 0.5], [:G2, 0.5]
]

bass_line_f = [
  [:F2, 0.5], [:C3, 0.5], [:F2, 0.5], [:C3, 0.5]
]

bass_line_f_variation = [
  [:F2, 0.5], [:A2, 0.25], [:C3, 0.25], [:F2, 0.5], [:C3, 0.5]
]

bass_line_g = [
  [:G2, 0.5], [:D3, 0.5], [:G2, 0.5], [:D3, 0.5]
]

# Rock expression functions with distortion
define :play_with_rock_expression do |note, duration, amplitude_base = 0.9|
  rock_amplitude =
  amplitude_base * rrand(0.9, 1.1)  # More aggressive variation
  rock_release = duration * rrand(0.7, 0.9)  # Longer sustain for rock
  with_fx :distortion, distort: 0.3 do
    play note, release: rock_release, amp: rock_amplitude
  end
  sleep duration
end

define :play_power_chord do |root, duration, amplitude_level = 0.8|
  with_fx :distortion, distort: 0.4 do
    play root, release: duration, amp: amplitude_level * 0.8
    play root + 7, release: duration, amp: amplitude_level * 0.6  # Fifth
  end
  sleep duration
end

# Rock transition functions
define :transition_to_subdominant_rock do
  with_fx :reverb, room: 0.6 do
    with_fx :distortion, distort: 0.2 do
      play_pattern_timed [[:E4, 0.25], [:F4, 0.25], [:G4, 0.5]], 0.25
      play :C5, release: 1.0, amp: 1.0
      sleep 1
    end
```

```
  end
end

define :transition_to_tonic_rock do
  with_fx :reverb, room: 0.6 do
    with_fx :distortion, distort: 0.2 do
      play_pattern_timed [[:F4, 0.25], [:E4, 0.25], [:D4, 0.5]], 0.25
      play :G4, release: 1.0, amp: 1.0
      sleep 1
    end
  end
end

# Conductor for proper synchronization
live_loop :rock_conductor do
  # Riff A
  2.times do
    cue :rock_strain_a
    sleep 4  # Duration of one riff A pattern
  end

  # Riff B
  2.times do
    cue :rock_strain_b
    sleep 4  # Duration of one riff B pattern
  end

  # Riff C
  2.times do
    cue :rock_strain_c
    sleep 4  # Duration of one riff C pattern
  end

  cue :rock_outro
  stop
end

# Rock introduction - establishing driving rhythm
live_loop :rock_intro do
  with_fx :distortion, distort: 0.2 do
    2.times do
      play_pattern_timed [[:C4, 0.25], [:E4, 0.25], [:G4, 0.5], [:E4, 0.25],
      [:C4, 0.25]], 0.25
      sleep 0.5
    end
  end
  stop
end

live_loop :bass_intro_rock do
  use_synth :fm  # Electric bass sound
  4.times do
    play :C2, release: 0.6, amp: 0.8
    sleep 0.5
    play :G2, release: 0.3, amp: 0.6
    sleep 0.5
  end
  stop
end
```

```
153
154 # Rock drums
155 live_loop :rock_drums do
156   sample :drum_bass_hard, amp: 1.2  # Kick on 1 and 3
157   sleep 1
158   sample :drum_snare_hard, amp: 1.0  # Snare on 2 and 4
159   sleep 1
160 end
161
162 live_loop :rock_hihat do
163   8.times do
164     sample :drum_cymbal_closed, amp: 0.4
165     sleep 0.5
166   end
167 end
168
169 # Drum fills for rock authenticity
170 live_loop :drum_fills do
171   sync :rock_conductor
172   if one_in(4)  # Random fills
173     sample :drum_tom_hi_hard, amp: 0.8
174     sleep 0.25
175     sample :drum_tom_mid_hard, amp: 0.8
176     sleep 0.25
177   end
178 end
179
180 sleep 4  # Wait for introduction to complete
181
182 # Main rock composition structure
183 live_loop :rock_bass do
184   use_synth :fm  # Electric bass
185   with_fx :reverb, room: 0.4 do
186     with_fx :distortion, distort: 0.1 do
187       # Riff A - Main key
188       2.times do |i|
189         sync :rock_conductor
190         if i == 0
191           bass_line_c.each do |note, duration|
192             play note, release: duration * 0.6, amp: 0.8
193             sleep duration
194           end
195         else
196           bass_line_c_variation.each do |note, duration|
197             play note, release: duration * 0.6, amp: 0.8
198             sleep duration
199           end
200         end
201       end
202
203       # Transition to subdominant
204       transition_to_subdominant_rock
205
206       # Riff B - Subdominant key
207       2.times do |i|
208         sync :rock_conductor
209         if i == 0
210           bass_line_f.each do |note, duration|
211             play note, release: duration * 0.6, amp: 0.8
```

```
            sleep duration
          end
        else
          bass_line_f_variation.each do |note, duration|
            play note, release: duration * 0.6, amp: 0.8
            sleep duration
          end
        end
      end

      # Transition back to tonic
      transition_to_tonic_rock

      # Riff C - Return to main key
      2.times do |i|
        sync :rock_conductor
        bass_line_c.each do |note, duration|
          play note, release: duration * 0.6,
          amp: 0.8 + (i * 0.15)  # Stronger crescendo
          sleep duration
        end
      end
    end
  end
end

live_loop :rock_guitar do
  use_synth :guit_e_fifths  # Electric guitar
  with_fx :reverb, room: 0.3 do
    with_fx :distortion, distort: 0.4 do
      # Riff A - Main theme
      2.times do |i|
        sync :rock_conductor
        if i == 0
          guitar_riff_a.each_with_index do |(note, duration), idx|
            progress = idx.to_f / guitar_riff_a.length
            current_amplitude = 0.8 + (progress * 0.4)  # Stronger crescendo
            play_with_rock_expression(note, duration, current_amplitude)
          end
        else
          guitar_riff_a_variation.each_with_index do |(note, duration), idx|
            progress = idx.to_f / guitar_riff_a_variation.length
            current_amplitude = 0.8 + (progress * 0.4)
            # Add power chords on climactic notes
            if idx == 5 || idx == 11
              play_power_chord(note - 12, duration, current_amplitude)
            else
              play_with_rock_expression(note, duration, current_amplitude)
            end
          end
        end
        sleep 0.5  # Rest at end of phrase
      end

      # Riff B - Subdominant section
      2.times do |i|
        sync :rock_conductor
        if i == 0
          guitar_riff_b.each_with_index do |(note, duration), idx|
```

```
271             progress = idx.to_f / guitar_riff_b.length
272             current_amplitude = 0.8 + (progress * 0.4)
273             play_with_rock_expression(note, duration, current_amplitude)
274           end
275         else
276           guitar_riff_b_variation.each_with_index do |(note, duration), idx|
277             progress = idx.to_f / guitar_riff_b_variation.length
278             current_amplitude = 0.8 + (progress * 0.4)
279             if idx == 5 || idx == 11
280               play_power_chord(note - 12, duration, current_amplitude)
281             else
282               play_with_rock_expression(note, duration, current_amplitude)
283             end
284           end
285         end
286         sleep 0.5  # Rest at end of phrase
287       end
288
289       # Riff C - Final strain
290       2.times do |i|
291         sync :rock_conductor
292         if i == 0
293           guitar_riff_c.each_with_index do |(note, duration), idx|
294             progress = idx.to_f / guitar_riff_c.length
295             current_amplitude = 0.8 + (progress * 0.5)  # Strongest crescendo
296             play_with_rock_expression(note, duration, current_amplitude)
297           end
298         else
299           guitar_riff_c_variation.each_with_index do |(note, duration), idx|
300             progress = idx.to_f / guitar_riff_c_variation.length
301             current_amplitude = 0.8 + (progress * 0.5)
302             # More power chords in final variation
303             if idx == 5 || idx == 8 || idx == 11
304               play_power_chord(note - 12, duration, current_amplitude)
305             else
306               play_with_rock_expression(note, duration, current_amplitude)
307             end
308           end
309         end
310         sleep 0.5  # Rest at end of phrase
311       end
312     end
313   end
314 end
315
316 # Rock outro with power chords - no ritardando
317 live_loop :rock_outro do
318   sync :rock_conductor  # Wait for conductor signal
319
320   use_synth :guit_e_fifths
321   with_fx :distortion, distort: 0.5 do
322     with_fx :reverb, room: 0.8 do
323       # Final power chord progression
324       play_power_chord(:C3, 2.0, 1.2)
325       play_power_chord(:G3, 1.5, 1.0)
326       play_power_chord(:C3, 3.0, 1.5)
327       sleep 4
328     end
329   end
```

```
330
331   # Final crash
332   sample :drum_cymbal_crash, amp: 1.0
333   sleep 2
334
335   # Stop all loops gracefully
336   stop
337 end
338
339 # MIDI output capability
340 define :send_midi_note do |note, duration, velocity = 100|
341   midi note, sustain: duration, vel: velocity
342   sleep duration
343 end
```

A.3 Feedback

Prompt:

BUG:
Runtime Error: [workspace_zero] - Thread death +–> :live_loop_rock_intro
Unknown synth :guit_e_fifths (RuntimeError)
ADD:
add more instruments

Code:

```
1  use_bpm 110
2
3  # Redefined chords for better harmonic progression
4  chords_for_a = [chord(:c3, :major), chord(:g3, :major)] # I–V progression
5  chords_for_b = [chord(:f3, :major), chord(:c3, :major)] # IV–I progression
6  chords_for_c = [chord(:f3, :major),
7  chord(:g3, :dominant7)] # IV–V7 progression
8  chords_for_d = [chord(:c3, :major), chord(:a3, :minor),
9  chord(:f3, :major), chord(:g3, :dominant7)] # I–vi–IV–V7
10
11 # Introduction (4 bars)
12 intro_melody = [:c5, :e5, :g5, :e5, :c5, :e5, :g5, :e5]
13 intro_rhythm = [0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5]
14
15 # Strain A – Main theme (16 bars)
16 strain_a_melody = [
17   :g4, :a4, :g4, :e4, :g4, :a4, :g4, :e4,
18   :d4, :e4, :d4, :b3, :c4, :d4, :e4, :f4,
19   :g4, :a4, :g4, :e4, :g4, :a4, :g4, :e4,
20   :d4, :f4, :a4, :g4, :f4, :e4, :d4, :c4
21 ]
22
23 strain_a_rhythm = [
24   0.75, 0.25, 0.5, 0.5, 0.75, 0.25, 0.5, 0.5,
25   0.75, 0.25, 0.5, 0.5, 0.25, 0.25, 0.25, 0.25,
26   0.75, 0.25, 0.5, 0.5, 0.75, 0.25, 0.5, 0.5,
27   0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 1.0, 1.0
28 ]
29
30 # Strain B – Contrasting theme (16 bars)
31 strain_b_melody = [
32   :f4, :g4, :a4, :g4, :f4, :e4, :d4, :c4,
```

```
    :f4, :g4, :a4, :g4, :f4, :e4, :d4, :c4,
    :a4, :bb4, :c5, :bb4, :a4, :g4, :f4, :e4,
    :d4, :e4, :f4, :g4, :a4, :g4, :f4, :e4
  ]

  strain_b_rhythm = [
    0.5, 0.5, 0.75, 0.25, 0.5, 0.5, 0.5, 0.5,
    0.5, 0.5, 0.75, 0.25, 0.5, 0.5, 0.5, 0.5,
    0.5, 0.5, 0.75, 0.25, 0.5, 0.5, 0.5, 0.5,
    0.5, 0.5, 0.5, 0.5, 0.75, 0.25, 0.5, 0.5
  ]

  # Strain C - Trio section (16 bars)
  strain_c_melody = [
    :c5, :d5, :e5, :f5, :e5, :d5, :c5, :b4,
    :c5, :d5, :e5, :f5, :e5, :d5, :c5, :b4,
    :a4, :b4, :c5, :d5, :c5, :b4, :a4, :g4,
    :a4, :b4, :c5, :d5, :e5, :d5, :c5, :b4
  ]

  strain_c_rhythm = [
    0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.5, 0.5,
    0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.5, 0.5,
    0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.5, 0.5,
    0.25, 0.25, 0.25, 0.25, 0.75, 0.25, 0.5, 0.5
  ]

  # Strain D - Final theme (16 bars)
  strain_d_melody = [
    :g4, :f4, :e4, :d4, :c4, :d4, :e4, :f4,
    :g4, :a4, :g4, :f4, :e4, :d4, :c4, :b3,
    :c4, :d4, :e4, :f4, :g4, :a4, :b4, :c5,
    :d5, :c5, :b4, :a4, :g4, :f4, :e4, :d4
  ]

  strain_d_rhythm = [
    0.75, 0.25, 0.5, 0.5, 0.75, 0.25, 0.5, 0.5,
    0.75, 0.25, 0.5, 0.5, 0.75, 0.25, 0.5, 0.5,
    0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25,
    0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 1.0, 1.0
  ]

  # Coda/Outro (4 bars)
  coda_melody = [:c5, :g4, :e4, :c4]
  coda_rhythm = [0.5, 0.5, 0.5, 1.5]

  # Improved chord progression using cue/sync
  live_loop :chord_progression do
    sync :right_hand

    # Strain A & Reprise
    2.times do
      cue :chord_change, chords: chords_for_a, pattern: :oom_pah
      sleep 4
    end

    # Strain B
    cue :chord_change, chords: chords_for_b, pattern: :waltz
    sleep 4
```

```
92
93    # Strain C (Trio)
94    cue :chord_change, chords: chords_for_c, pattern: :arpeggio
95    sleep 4
96
97    # Strain D
98    cue :chord_change, chords: chords_for_d, pattern: :oom_pah
99    sleep 4
100 end
101
102 # Varied left hand patterns
103 live_loop :left_hand do
104   use_synth :piano
105   current_chords, current_pattern = sync :chord_change
106
107   case current_pattern
108   when :oom_pah
109     8.times do # Play for 2 bars
110       current_chords.each do |c|
111         play c[0], amp: 0.8, release: 0.4 # Root
112         sleep 0.5
113         play c, amp: 0.6, release: 0.3 # Full chord
114         sleep 0.5
115       end
116     end
117   when :waltz
118     8.times do # Play for 2 bars
119       current_chords.each do |c|
120         play c[0], amp: 0.7, release: 0.5
121         sleep 1.0
122         play c, amp: 0.5, release: 0.4
123         sleep 0.5
124         play c[2], amp: 0.5, release: 0.4 # Fifth
125         sleep 0.5
126       end
127     end
128   when :arpeggio
129     # A broken chord pattern for the trio
130     4.times do
131       current_chords.each do |c|
132         3.times do |i|
133           play c[i], amp: 0.6, release: 0.2, cutoff: rrand(0.7, 0.9)
134           sleep 0.25
135         end
136         sleep 0.25
137       end
138     end
139   end
140 end
141
142 # Right hand melody with improved variation
143 live_loop :right_hand do
144   use_synth :piano
145
146   # Introduction
147   with_fx :level, amp: 0.8 do
148     intro_melody.each_with_index do |note, idx|
149       play note, amp: 0.7, release: 0.3
150       sleep intro_rhythm[idx]
```

```
    end
  end

  # Strain A (first time)
  with_fx :level, amp: 0.9 do
    strain_a_melody.each_with_index do |note, idx|
      play note, amp: 0.8, release: 0.2
      sleep strain_a_rhythm[idx]
    end
  end

  # Strain A (second time with slight variation)
  with_fx :level, amp: 0.95 do
    with_fx :echo, phase: 0.25, decay: 2 do
      strain_a_melody.each_with_index do |note, idx|
        play note, amp: 0.85, release: 0.25
        sleep strain_a_rhythm[idx]
      end
    end
  end

  # Strain B (repeats) - softer dynamics with reverb
  2.times do
    with_fx :reverb, room: 0.4 do
      with_fx :level, amp: 0.7 do
        strain_b_melody.each_with_index do |note, idx|
          play note, amp: 0.6, release: 0.2
          sleep strain_b_rhythm[idx]
        end
      end
    end
  end

  # Strain A reprise (octave higher for variation)
  with_fx :level, amp: 1.0 do
    strain_a_melody.each_with_index do |note, idx|
      play note + 12, amp: 0.9, release: 0.2 # +12 adds an octave
      sleep strain_a_rhythm[idx]
    end
  end

  # Strain C - Trio (repeats, different synth)
  2.times do
    use_synth :prophet
    with_fx :reverb, room: 0.8 do
      with_fx :level, amp: 1.0 do
        strain_c_melody.each_with_index do |note, idx|
          # Add a slight crescendo through the phrase
          current_amp = 0.8 + (idx.to_f / strain_c_melody.length) * 0.2
          play note, amp: current_amp, release: 0.3, cutoff: rrand(0.8, 0.95)
          sleep strain_c_rhythm[idx]
        end
      end
    end
    use_synth :piano
  end

  # Transition before Strain D
  with_fx :level, amp: 1.1 do
```

```
210       play_pattern_timed [:g4, :a4, :b4, :c5, :d5, :e5, :f5, :g5],
211       [0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.25]
212       play chord(:g3, :dominant7), amp: 1.0, release: 1.5
213       sleep 1.5
214       sleep 0.5 # Brief silence for dramatic effect
215     end
216
217     # Strain D (repeats)
218     2.times do
219       with_fx :level, amp: 0.9 do
220         strain_d_melody.each_with_index do |note, idx|
221           play note, amp: 0.8, release: 0.2
222           sleep strain_d_rhythm[idx]
223         end
224       end
225     end
226
227     # Coda/Outro with ritardando effect
228     with_fx :level, amp: 0.8 do
229       use_bpm 100 # Slow down for ending
230       coda_melody.each_with_index do |note, idx|
231         play note, amp: 0.7, release: 0.5
232         sleep coda_rhythm[idx]
233       end
234     end
235
236     # Final chord with reverb
237     use_bpm 110
238     with_fx :reverb, room: 1.0 do
239       play chord(:c4, :major), amp: 1.0, release: 3.0
240       sleep 3.0
241     end
242   end
```